



Classes e Objetos

Aula 05 · Bloco 2 — Os Pilares da POO

Classe é a planta. Objeto é a construção.

Nesta aula

1. O problema que a POO resolve
2. **Classe** × **objeto**
3. Sua primeira classe
4. `new`, referência e `null`
5. **Construtores** e `this`

A POO faz outra pergunta

Você vinha perguntando:

"quais dados eu preciso guardar?"

A POO pergunta:

**"quais coisas existem no meu problema,
e o que cada uma sabe e sabe fazer?"**

No problema do boletim

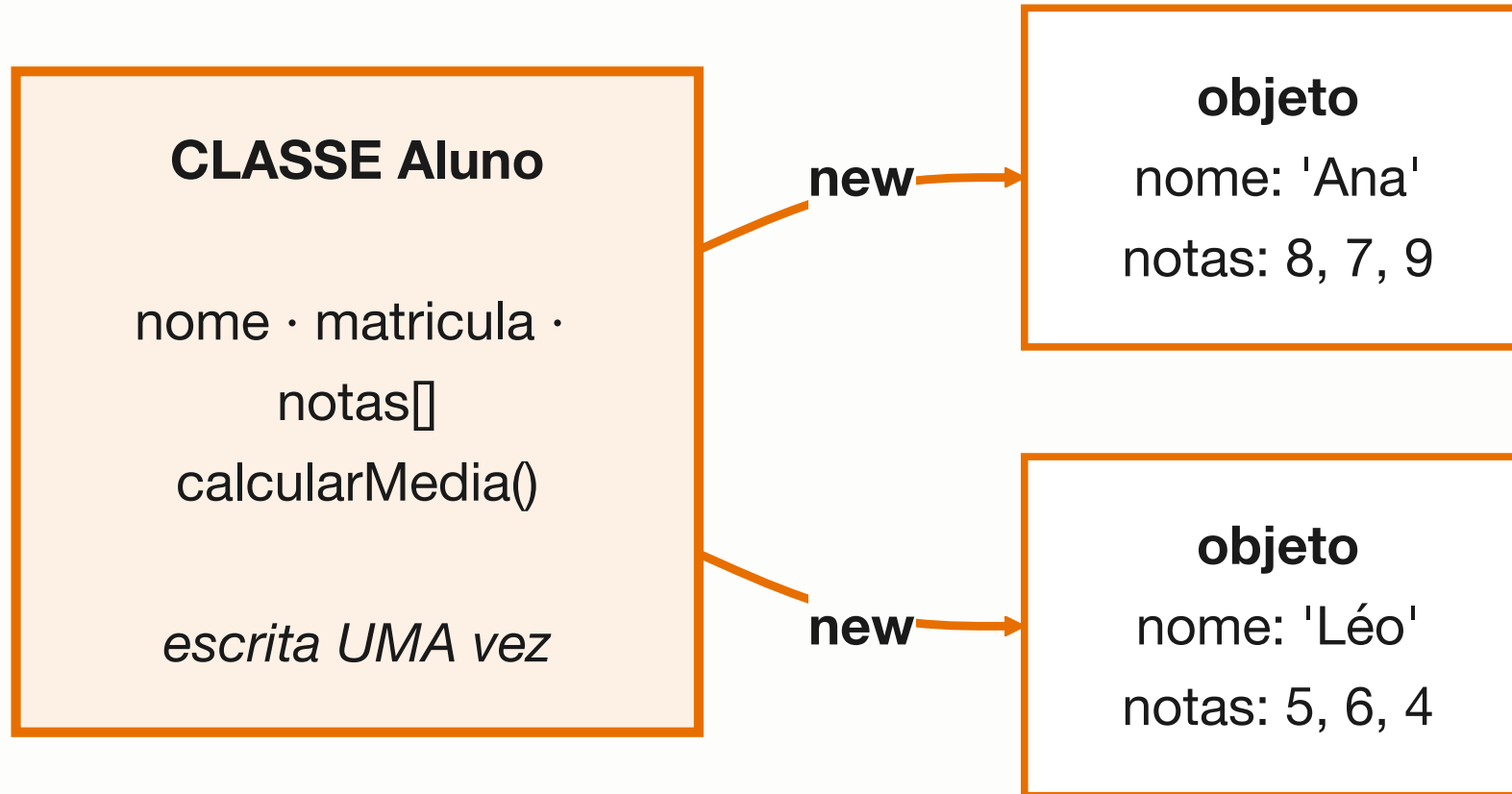
Existe uma coisa chamada **aluno**.

Ele **sabe** um nome, uma matrícula e algumas notas.

E ele **sabe fazer** uma coisa: calcular a própria média.

Dados e comportamento **juntos** — isto é o objeto.

Classe é a planta; objeto é a construção



Sua primeira classe

```
public class Aluno {  
    String nome;                // ATRIBUTOS: o que ele SABE  
    double[] notas = new double[3];  
  
    double calcularMedia() {    // MÉTODOS: o que ele SABE FAZER  
        double soma = 0;  
        for (double nota : notas) soma += nota;  
        return soma / notas.length;  
    }  
    boolean estaAprovado() { return calcularMedia() >= 7; }  
}
```

Sem `main` — não é um programa, é um modelo.

👁️ Repare no que sumiu

Nenhum método recebe o array de notas por parâmetro.

`calcularMedia()` **já sabe** de quais notas está falando: as do **próprio objeto**.

Essa é a diferença entre um método `static` solto e um método de instância.

Usando a classe

```
public class Escola {  
    public static void main(String[] args) {  
        Aluno ana = new Aluno();           // cria um objeto na memória  
        ana.nome = "Ana";                  // o ponto acessa o que é do objeto  
        ana.notas[0] = 8.0;  
  
        Aluno leo = new Aluno();           // outro objeto, independente  
        leo.nome = "Léo";  
  
        ana.imprimirBoletim();  
    }  
}
```


Onde foi parar o `static` ?

`calcularMedia()` **não** é `static` porque depende de **qual** aluno.

`main` é `static` porque precisa existir antes de qualquer objeto.

Chamar método de instância direto do `main`, sem objeto:

```
error: non-static method cannot be referenced  
      from a static context
```

💡 O compilador está perguntando: *"a média de quem?"*

new e referência

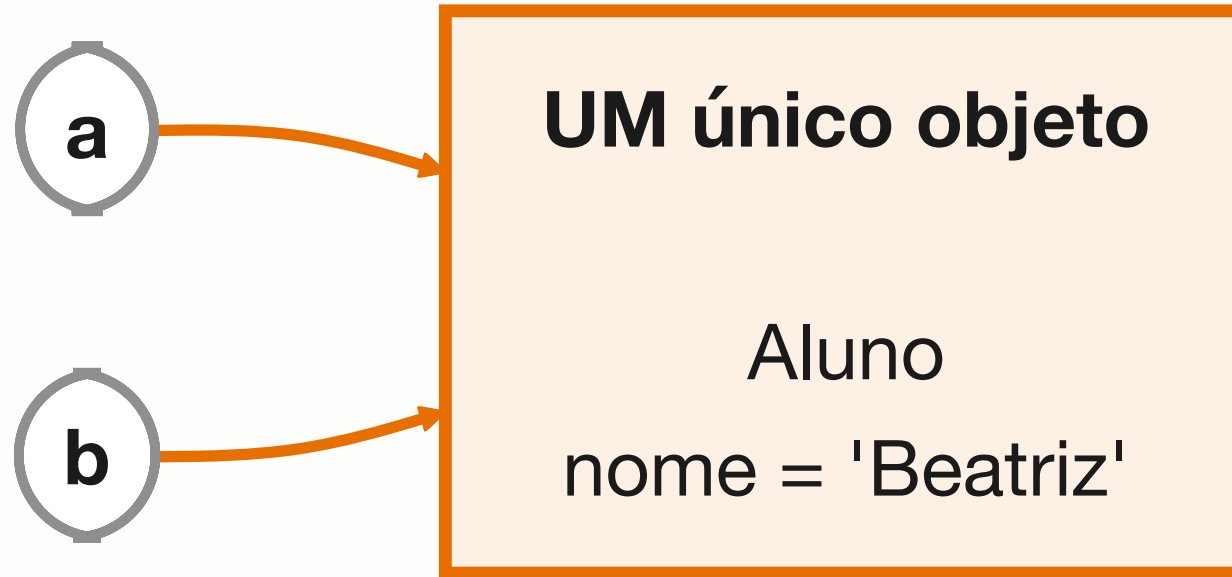
```
Aluno ana = new Aluno();
```

Três coisas numa linha: `new Aluno()` cria o objeto, `Aluno ana` declara a variável, e o `=` guarda o endereço nela.

A variável **não guarda o objeto** — guarda uma **referência** a ele.

```
Aluno b = a;           // NÃO copia: dá um segundo nome ao MESMO objeto
b.nome = "Beatriz";
System.out.println(a.nome); // Beatriz 🤖
```

Um objeto, dois apelidos





NullPointerException

```
Aluno c = null;  
c.nome;           //
```

O erro de execução **mais comum do Java**.

E a causa é quase sempre a mesma:

faltou um new — ou uma busca não encontrou nada.

Construtores: nascer já pronto

```
public class Aluno {  
    String nome;  
  
    // mesmo nome da classe, SEM tipo de retorno (nem void!)  
    public Aluno(String nome, String matricula) {  
        this.nome = nome;        // this.nome = atributo; nome = parâmetro  
        this.matricula = matricula;  
    }  
}
```

Agora `new Aluno()` **não compila**: objeto pela metade deixou de ser possível.

O que é **this**

A referência ao **objeto atual** — "eu mesmo".

Obrigatório quando o parâmetro tem o mesmo nome do atributo:

```
this.nome = nome;    // "o MEU nome recebe o nome que veio de fora"  
nome = nome;         // X o parâmetro atribui a si mesmo; o atributo fica null
```

⚠ Ao escrever qualquer construtor, o construtor vazio deixa de existir. Se quiser os dois, declare os dois — assunto da aula 06.

O pulo do gato

Objetos cabem num array **como qualquer outro valor**:

```
Aluno[] turma = new Aluno[3];
turma[0] = new Aluno("Ana", "1001");
turma[1] = new Aluno("Léo", "1002");

for (Aluno aluno : turma) {
    aluno.imprimirBoletim();    // cada um usa os PRÓPRIOS dados
}
```

Um array. Três objetos completos. Zero sincronização manual.

Compare com o desafio da aula 04.

Exercícios da aula

Na pasta `aula-05/`, cada classe em seu arquivo:

1. **Livro** + **Estante** — `emprestar()`, `devolver()`, `exibirFicha()` ;
2. **ContaBancaria** + **Banco** — `depositar`, `sacar` com aviso, `exibirExtrato` ;
3. **Referencia.java** — duas variáveis, um objeto. Explique o resultado. E provoque um `NullPointerException` ;
4. **Retangulo** + **Geometria** — construtor, `calcularArea()`, `ehQuadrado()` ;
5. **Desafio** 🌶️ — refaça o boletim da aula 04 com classe. **Quantas linhas** cada versão tem?

Próxima aula

Aula 06 — Encapsulamento

O `+` do diagrama significa `public`.

Você vai descobrir por que deixar tudo público é péssima ideia.